

Examen Parcial React – Programación 4

Estructura de archivos básica

(Puede agregar más archivos si lo desea)

src/modelo/Producto.ts

src/paginas/lista_productos.tsx

src/paginas/lista_compra_final.tsx

src/context/compra_context.ts

Realice mediante TypeScript

Una familia dispone de un presupuesto mensual fijo de:

\$250.000

y debe realizar las compras necesarias para cubrir las necesidades básicas del hogar.

El supermercado proporciona un catálogo de productos agrupados en categorías.

El usuario deberá seleccionar productos respetando el presupuesto y asegurando que todas las necesidades básicas estén cubiertas.

La lista de artículos disponibles se encuentra en el archivo adjunto productos_super.json

Se solicita desarrollar una aplicación React que permita buscar y cargar los artículos deseados respetando el presupuesto disponible y los requisitos mínimos por categoría.

Cada **Producto** deberá estar representado por el objeto correspondiente

Clase Producto

Cada producto estará representado mediante la siguiente clase:

```
class Producto {  
  codigoProducto: string;  
  denominacion: string;  
  categoria: string;  
  codigoCategoria: string;  
  precioVenta: number;  
  cantidad: number;  
}
```

Inicialmente todos los productos del JSON se cargan con: cantidad 0

Página Principal

La página principal será:

lista_productos.tsx la cual debe presentar 2 columnas

Columna Izquierda

Productos disponibles.

Presupuesto Disponible: \$250.000

A medida que se compren artículos, el presupuesto deberá actualizarse automáticamente, decrementando su valor.

Listado de artículos

Debajo del presupuesto deberá mostrarse:

un input type="text" que permita ingresar el código del producto que se desea comprar

Debajo del input una grilla con todos los artículos disponibles para comprar.

Cada fila de la grilla deberá mostrar:

Código Producto	Denominación	Categoría	Precio Venta
--------------------	--------------	-----------	--------------

Columna Derecha

Carrito de compras.

En esta columna se deberán presentar la grilla de productos que se vayan cargando de la lista de productos disponibles con las columnas

Código Producto	Denominación	Categoría	Precio Venta	Cantidad	Subtotal
--------------------	--------------	-----------	-----------------	----------	----------

En la parte superior de la grilla se debe mostrar el Total de la compra que será el equivalente a la suma de todas las filas (productos) agregados a la grilla

Debajo de la Grilla debe agregar un botón **"Finalizar Compra"** que da por terminada la compra actual

Lógica de la interface

El usuario ingresa el código del producto que quiere adquirir, el sistema busca el producto por el código, si lo encuentra lo agrega a la lista de productos comprados y calcula el total. Si el producto ya fue agregado anteriormente se deberá incrementar solo la cantidad y recalcular el total.

Si el código ingresado no existe deberá avisar al usuario **“El código ingresado no es válido, intente nuevamente”**.

El usuario también debe tener la posibilidad de eliminar un producto de la grilla de productos comprados si lo desea y recalcular el total.

Finalmente si el usuario presiona el botón **“Finalizar Compra”**, se deberá validar que la compra cumple con los requisitos del presupuesto familiar y si es así cargar la interface “lista_compra_final.tsx”

Interface Lista Compra Final (lista_compra_final.tsx)

Debe presentar la grilla con los productos comprados y el total calculado

Variables principales

Las variables deberán administrarse utilizando:

useContext

en el archivo:

compra_context.ts

Variables

productosDisponibles: Producto[]

productosSeleccionados: Producto[]

presupuestoInicial: number

montoGastado: number

montoDisponible: number

Validaciones

La compra deberá cumplir:

Cantidad total

Al menos:

15 productos

Requisitos por categoría

Controle que se agregue al menos 1 producto de cada categoría

Presupuesto

El presupuesto disponible nunca podrá ser negativo.

Mensaje de error

Si alguna validación falla deberá mostrarse el mensaje:

La compra no cubre las necesidades básicas o excede el presupuesto disponible.

Ejemplo de Interfaces

PRODUCTOS DISPONIBLES
Presupuesto Disponible: \$250.000
Ingresar Código Producto (Ej: 1252) **Buscar y Comprar**

Código Producto	Denominación	Categoría	Precio Venta
1252	Arroz 1Kg	Almacén	\$6000
1253	Fideos Tallarín	Almacén	\$4500
2001	Leche Entera 1L	Lácteos	\$3800
2002	Yogur Natural	Lácteos	\$3200
3010	Pan Francés	Panadería	\$5200
4101	Manzanas (1kg)	Verdulería	\$7500
5005	Papel Higiénico	Limpieza	\$9800

CARRITO DE COMPRAS (Artículos Seleccionados)
Total de la compra: \$0

Código Producto	Denominación	Categoría	Precio Venta	Cantidad	Subtotal
-----------------	--------------	-----------	--------------	----------	----------

Finalizar Compra

Página Final

**La compra cumple los requisitos y cubre las necesidades básicas.**
Total Invertido: \$87.500
Monto Restante: \$162.500

PRODUCTOS COMPRADOS (Total: 22 Artículos)

Código Producto	Denominación	Categoría	Precio Venta	Cantidad	Subtotal
1252	Arroz 1Kg	Almacén	\$6000	2	\$12000
1253	Fideos Tallarín	Almacén	\$4500	3	\$13500
2001	Leche Entera 1L	Lácteos	\$3800	4	\$15200
2002	Yogur Natural	Lácteos	\$3200	2	\$6400
3010	Pan Francés	Panadería	\$5200	2	\$10400
4101	Manzanas (1kg)	Verdulería	\$7500	2	\$15000
4102	Papas (1kg)	Verdulería	\$5500	1	\$5500
5005	Papel Higiénico	Limpieza	\$9800	1	\$9800
22 Artículos Comprados			\$87.500		

Aceptar

AYUDA TÉCNICA

El contexto debería contener como mínimo:

Variables

```
productosDisponibles: Producto[]  
productosSeleccionados: Producto[]  
presupuestoInicial: number  
montoGastado: number  
montoDisponible: number
```

Métodos

```
agregarProducto: (codigo: string) => void  
quitarProducto: (producto: Producto) => void  
puedeGenerarCompra: () => string | null
```

Posible implementación

Agregar producto

```
function agregarProducto(codigo: string): string | null {  
  const producto = productosDisponibles.find(  
    (p) => p.codigoProducto === codigo  
  )  
  if (!producto) {  
    return "El código ingresado no es válido, intente nuevamente"  
  }  
  if (producto.precioVenta > montoDisponible) {  
    return "La compra no cubre las necesidades básicas o excede el presupuesto disponible."  
  }  
  
  setProductosSeleccionados((prev) => {  
    const existente = prev.find(  
      (p) => p.codigoProducto === producto.codigoProducto  
    )  
  
    if (existente) {  
      return prev.map((p) =>  
        p.codigoProducto === producto.codigoProducto  
        ? { ...p, cantidad: p.cantidad + 1 }  
        : p  
      )  
    }  
  
    return [...prev, { ...producto, cantidad: 1 }]  
  })  
  
  return null  
}
```

Quitar producto

```
function quitarProducto(producto: Producto) {  
  setProductosSeleccionados((prev) => {  
    const existente = prev.find(  
      (p) => p.codigoProducto === producto.codigoProducto  
    )  
  
    if (!existente) return prev  
  
    if (existente.cantidad === 1) {  
      return prev.filter(  
        (p) => p.codigoProducto !== producto.codigoProducto  
      )  
    }  
  
    return prev.map((p) =>  
      p.codigoProducto === producto.codigoProducto  
        ? { ...p, cantidad: p.cantidad - 1 }  
        : p  
    )  
  })  
}
```

```
const montoGastado = productosSeleccionados.reduce(  
  (total, producto) =>  
    total + (producto.precioVenta * producto.cantidad),  
  0  
);
```

```
const cantidadTotal = productosSeleccionados.reduce(  
  (total, producto) =>  
    total + producto.cantidad,  
  0  
);
```